

Status Report 1
Group 1
March 20, 2025

Introduction

In the first iteration, our primary plan was to get a basic web server locally, then have part of our group design sketches of the user interface for Grade Gator and begin building the basic structure of our site on React while the other half worked on developing the backend with Django. We separated the web server into four Django apps: courses, assignments, accounts, and grading. Each of these apps contains a set of top-level API endpoints. See the screenshots section of the report for a complete list of endpoints. We set up a swagger auto documentation of the API using the `drf_spectacular` Django module.

We created 3 Figma prototypes showcasing instructor and student views, focusing on log in, landing, courses, and assignment pages. As part of our first sprint and frontend work, we also aimed to standardize the frontend tech stack to ensure everyone was using the same tools to create our project, and that the tools were well understood. In terms of the work we completed with the back end, we set up a file storage system through Amazon Web Services which allows us to store files submitted on the frontend, by either an instructor or student, so that the backend can access and use them in the autograding process. We also developed a basic autograder that employs containers from Docker to run code files submitted by students in three different programming languages.

Changes

Maddie Creed became the product owner on the first day of the sprint. Due to technological difficulties, she could not run our initial servers on her computer. Therefore, we moved responsibilities around to ensure she can contribute by decreasing the need for her to access the servers.

Customer-Iteration Goals

The product backlog at the time this sprint started included:

- Create distinct user interfaces for student and instructor views.
- Create a gradebook that supports standards based grading
- Create a submission service that included the use of an autograder
- Set up file storage to allow files and information uploaded on the front end to be accessed for various tasks on the backend.
- Create containers for Java, Python, and C for the autograder to run student code submissions in.

- Set up a basic autograder that connects to Docker containers, finds student code submissions and runs alongside a JSON file the instructor has uploaded with the assignment rubric.
- Create a basic front end and back end structure that will run locally.
- Migrate the basic frontend and backend to run on a shared server.
- Host the frontend on a website rather than a local host.
- Organize the backend databases so necessary information is stored and can be easily accessed.
- Require login authentication.
- Research and implement ways to securely store student data once on a shared server.
- Include review of instructor comments and grading on the student view of assignments.
- Research the potential for Moodle integration of class rosters.
- Allow students to view rubrics for assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.
- Meet with professors in the math department to discuss needs of department assignments.

Our customer desired working software for this sprint. He did not specify if he preferred front or back end software more. Therefore, we worked on both by creating a basic webpage, designing prototypes, creating an autograder, and creating a code/pdf file upload and storage system.

The tasks in our sprint backlog for this iteration included:

1. Create 3 sketches of the user interface for student and instructor views. This includes the homepage, course page, and assignment page.
2. Set up file storage to allow files and information uploaded on the front end to be accessed for various tasks on the backend.
3. Create containers for Java, Python, and C, for the autograder to run student code submissions in.
4. Set up a basic autograder that connects to Docker containers, finds student code submissions and runs alongside a JSON file the instructor has uploaded with the assignment rubric.
5. Standardize the frontend tech stack so that all software developers are using the same tools and the tools are well defined on how to use them.
6. Create a basic front end and back end structure that will run locally.
7. Organize the backend databases so necessary information is stored and can be easily accessed.

For this iteration, we chose to focus on standardizing the frontend tech stack, starting to build a basic autograder, designing sketches for the user interface of our product, hosting a web server with basic functionality locally, and setting up file storage. We felt that building a functioning

basic backend would be integral to any part of the front end moving forward, which is why we focused mostly on backend development for this sprint. By making incremental progress in multiple components, namely, hosting, file storage, database management, and autograding function, we have set ourselves up to connect these pieces by the end of the next sprint, hopefully resulting in a basic, but fully functioning, software. We also used front end development as a way to illuminate backend considerations. For example, our work in the user interface was integral to determine the information that should be included in the databases on the backend.

Use Cases

We created use cases for our primary goals: uploading assignments for autograding, creating a new assignment as a professor, and storing student submitted files.

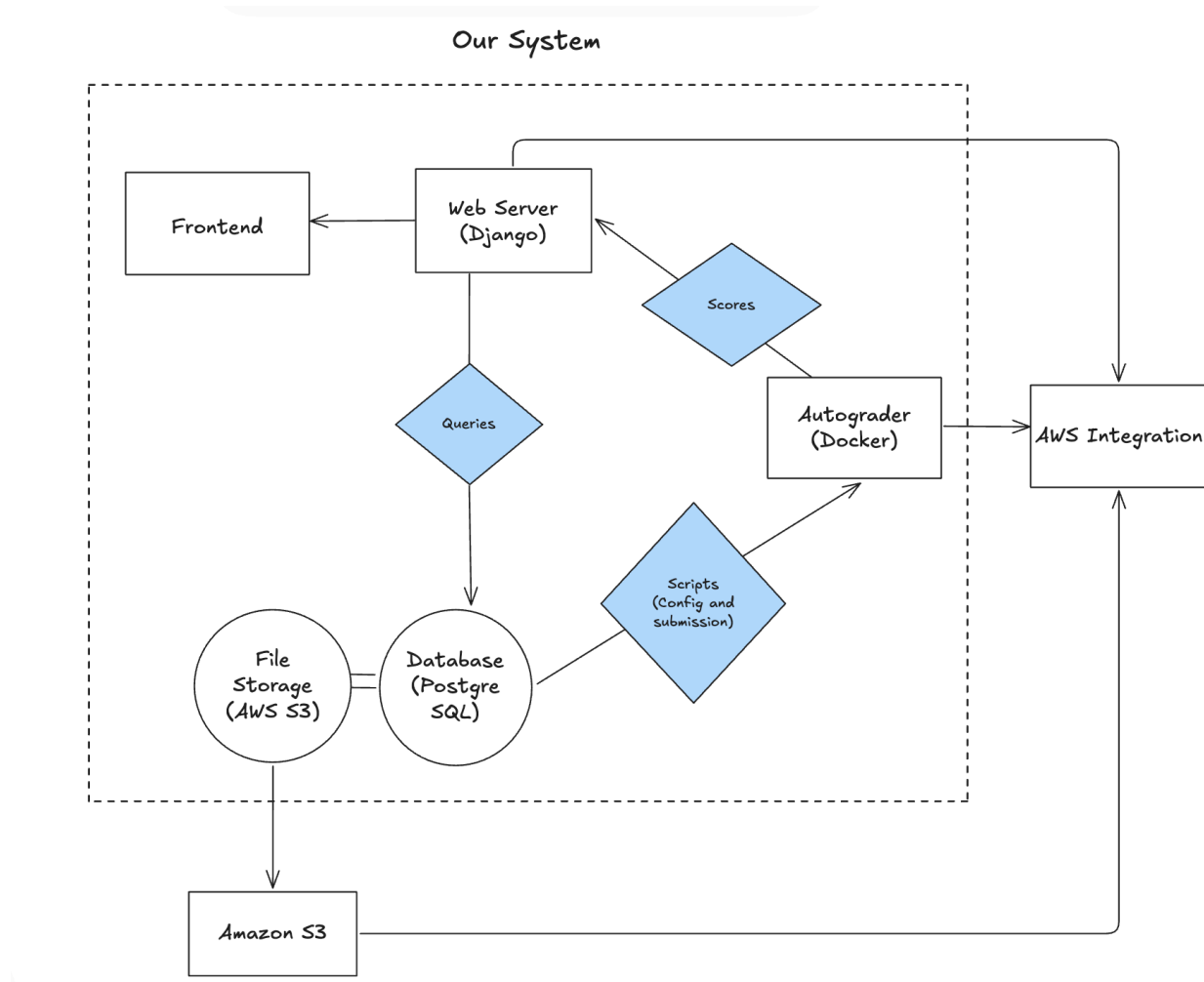
Name: Uploading assignments for auto-grading	
Goal: A student uploads the file from the local end, and the auto-grader grades it and gives back a score according to the rubric provided	
Actors: Student, Grade Gator	
Basic Flows: { Student selects course and assignment } 1. The student navigates to the course page from the dashboard. 2. The student selects the relevant assignment from the list of assignments. { Student uploads file } 3. The student selects the file from their device. 4. The student confirms the submission. { System grades the assignment } 5. The system reads the file and sends it to the auto-grader running in Docker. { System gives back the result } 6. The system receives a report from the auto-grader. 7. The system prints out the unit testing results on the screen.	Alternative Flows: <i>Specific alternative flow:</i> At { System grades the assignment }, if the auto-grader can't recognize the type of file students uploaded, the system displays "Auto-grader error! The type of file is not supported" and restart flow at { Student uploads file }. <i>Bounded alternative flow:</i> At any point of { Student uploads file }, if network access is lost, display "Network Failure. Please try submitting a file again". Restart flow at { Student uploads file }.

Name: Creating a new assignment
Goal: An instructor creates a new assignment for a certain course
Actors: Instructor, Grade Gator

Basic Flows: { Professor selects a course and starts creating assignments } 1. The professor selects the relevant course from the dashboard. 2. The professor clicks on the "Create Assignment" button. { Professor add related information for the assignment } 3. The professor puts in assignment information. 4. The professor sets up auto-graders and rubrics. { System saves the information } 5. The system stores the information in the database. { System presents the assignment } 6. The system displays the assignment on both the student's and professor's end.	Alternative Flows: <i>Specific alternative flow:</i> At { Professor add related information for the assignment }, if missing certain required information, such as due date, the system displays “Please fill in the required fields” and restart flow at { Professor add related information for the assignment }. <i>Bounded alternative flow:</i> At any point of { System saves the information }, if network access is lost, display “Network Failure. Please try again”. Restart flow at { Professor add related information for the assignment }.
--	---

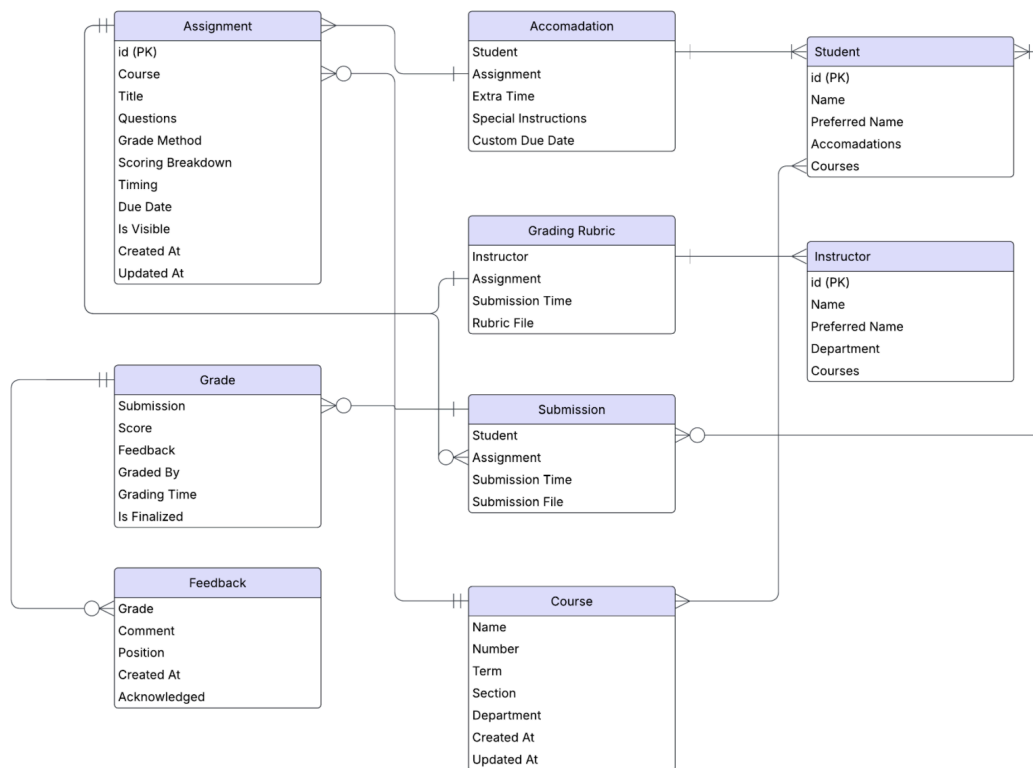
Name: Storing a Code/PDF/JSON file	
Goal: A student/instructor uploads a file from the local end, and the file is securely stored in the corresponding folder inside the AWS S3 bucket	
Actors: Student/Instructor, Grade Gator	
Basic Flows: { Student selects course and assignment } 1. The student navigates to the course page from the dashboard. 2. The student selects the relevant assignment from the list of assignments. { Student uploads the file } 3. The student selects the file from their device. 4. The student confirms the submission. { Grade Gator processes the assignment } 5. The system verifies the file format and size 6. The system sends the file to the corresponding folder inside the grade-gator-files S3 bucket. { Grade Gator stores the assignment }	Alternative Flows: <i>Specific alternative flow:</i> At { Grade Gator processes the assignment }, if the size of the file being uploaded exceeds the allowed size, the system displays “File size exceeds the limit. Please upload a valid file!” and restart flow at { Student uploads the file }. <i>Bounded alternative flow:</i> At any point of {Student uploads file }, if network access is lost, display “Network Failure. Please try submitting file again”. Restart flow at { Student uploads the file }.

System Description



The general system diagram remains unchanged. However, we have made updates to our database schema. The new Database Entity-Relationship Diagram below records the new components and attributes of the entities. We have added submission, feedback, and accommodation entities rather than having accommodation be an attribute of the student entity, or submission and feedback be an attribute of the assignment entity. This allows us to store multiple assignment submissions and feedback for each student and will make it easier for

instructors to add accommodations in their assignments.



Current Status

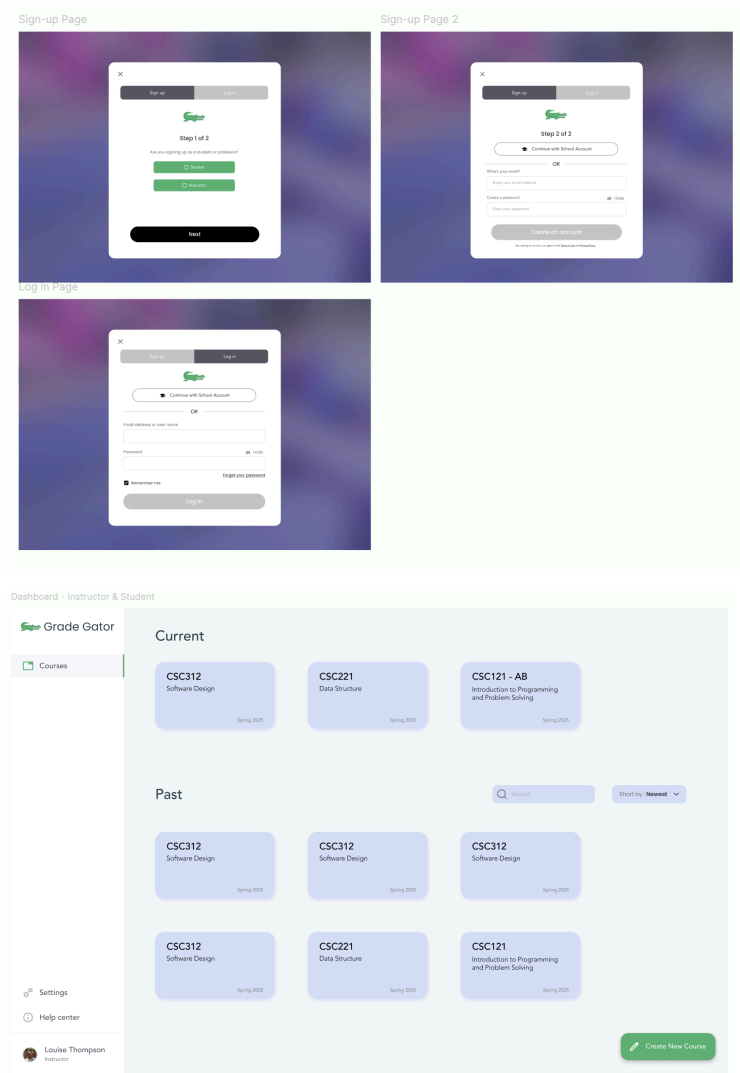
Our current project status is best understood by looking at each component, as summarized below:

- 1) We have designed multiple modules for our autograder with Django apps: Course, Student, Instructor, Assignment (title of the assignment, description of the assignment, deadlines, etc.), Submission (student name, assignment content, submission date, etc.), and StudentAccommodation. All these modules have corresponding sets of API endpoints (enables authentication).
- 2) We have utilized PostgreSQL as our database, and the database connection is implemented and the initial migration is run. The database successfully links the submission and assignments to the student and instructor, tracking every submission closely.
- 3) We have deployed an AWS S3 file storage system that makes file uploads and storages possible.
- 4) For the frontend, we have created three Figma prototypes for both student and instructor views. These prototypes can be used as templates for developers to finish up the backend coding tasks.

- 5) The web server is currently running locally and every developer can access the server as a superuser to test the functionality of our web server.

Screenshots

Frontend prototypes for UI development:



Grade Gator

Assignments

Grade Book

Settings

Help center

Louise Thompson
Student

CSC312 Software Design

Spring 2025

SEARCH

ASSIGNMENT NAME	STATUS	TIME CREATED	RUBRIC	DEADLINE
Homework 1	82 / 100	March 15 2021, 12:47 PM		March 31 2021, 11:59 PM
Homework 2	Submitted Late	March 15 2021, 12:47 PM		March 31 2021, 11:59 PM
Homework 3	Late	March 15 2021, 12:47 PM		March 31 2021, 11:59 PM
Homework 4	Submitted	March 15 2021, 12:47 PM		March 31 2021, 11:59 PM
Homework 4	Not Submitted	March 15 2021, 12:47 PM		March 31 2021, 11:59 PM

CSC121 Spring

My Courses

Dashboard

Profile

HL Prof.

Notifications

Assignment	Student	Type	Quick Action
Homework 1	Maddie Creed	Regrade Request	Approve
Homework 1	Katherine Hu	Regrade Request	Approve
Homework 2	Katherine Hu	Extension	Approve

Assignments

Name	Student View	Submissions	Graded	Status
Create New Assignment				
Homework 1	Visible	18 New	5	
Homework 2	Visible	12 New	0	Publish

CSC121 Spring

My Courses

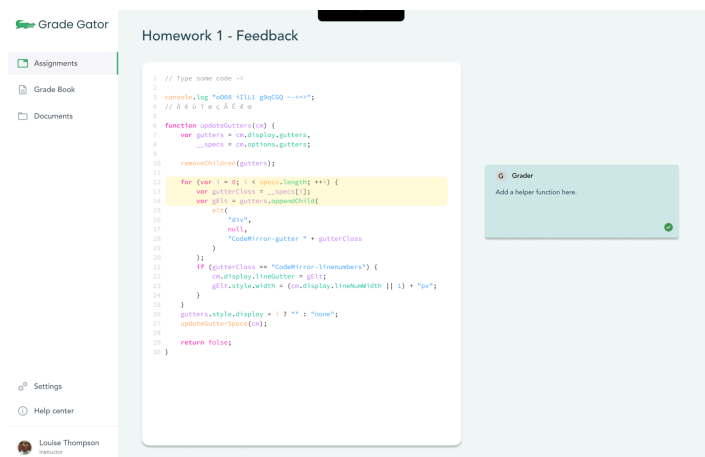
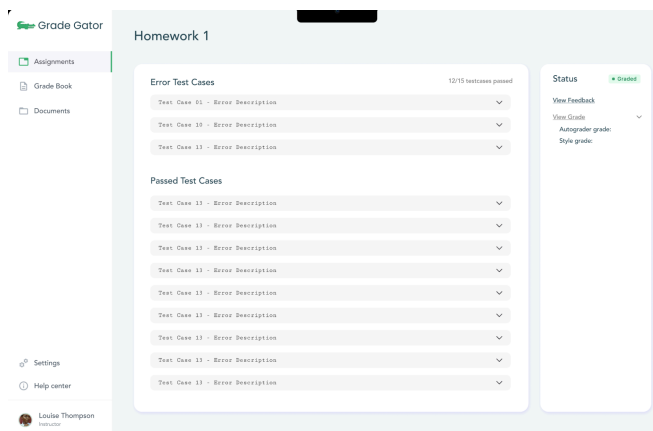
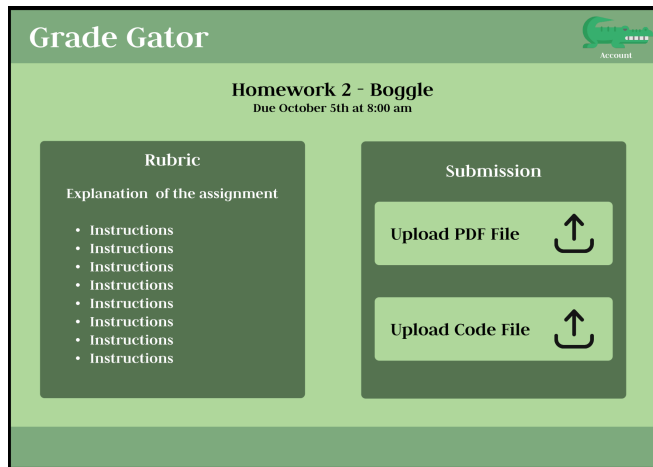
Dashboard

Profile

HL Prof.

Homework 1

Student Submissions	Autograder Score	Grade
Create New Submission		
Maddie Creed	10/12	Ungraded
Katherine Hu	12/12	15/15
Katherine Hu	Unsubmitted	Ungraded



Above are screenshots from each of our Figma prototypes. This is part of our front-end work where we try to showcase Grade Gator's functionality in a unique and intuitive way. The front-end design is important to ensure users can utilize Grade Gator easily and quickly. The Figma prototypes are linked here:

<https://www.figma.com/proto/MEFqDJzRM7ZTdDGCYOfa1p/Student-%26-Prof-Views?node-id=272-649&t=5DsBSDyK7T7xZhKL-1>

1. The first image is a collection of our signup/login page. Students or instructors can sign up using either their school account or other email addresses.
2. The second image is the landing page. It's been separated into current and past, where the "Current" section shows courses they registered/teaching this semester. The past section shows the courses they previously took/taught, and there's a search and sort function that helps the users to find the course quickly.
3. The third image shows the course dashboard of a student's view. They can view all the assignments from the dashboard and their status, time created, rubric, and deadline.
4. The fourth image shows the course dashboard of a professor's view. There's a notification section where they can see recent requests made by the student. Also, they can create and view assignments they created, see the number of submissions and assignments graded, and publish grades for assignments.
5. The fifth image compliments the fourth image to show the submission details of a certain assignment.
6. The sixth image is a student's view of the detail page for a certain assignment. The student can see the rubrics and upload either code/pdf submissions.
7. The seventh image demonstrates the results after running the auto-grader. It shows the number of test cases students passed and failed. By clicking the expand button, the students can read about more detailed error information. After being graded by graders, they have to first go through and click all the comments (as in the eighth image) before they view their scores.
8. The eighth image is a feedback page. To better help students understand the comments, the code that the graders referring to will be highlighted and they need to click through the comment similar to Google Doc.

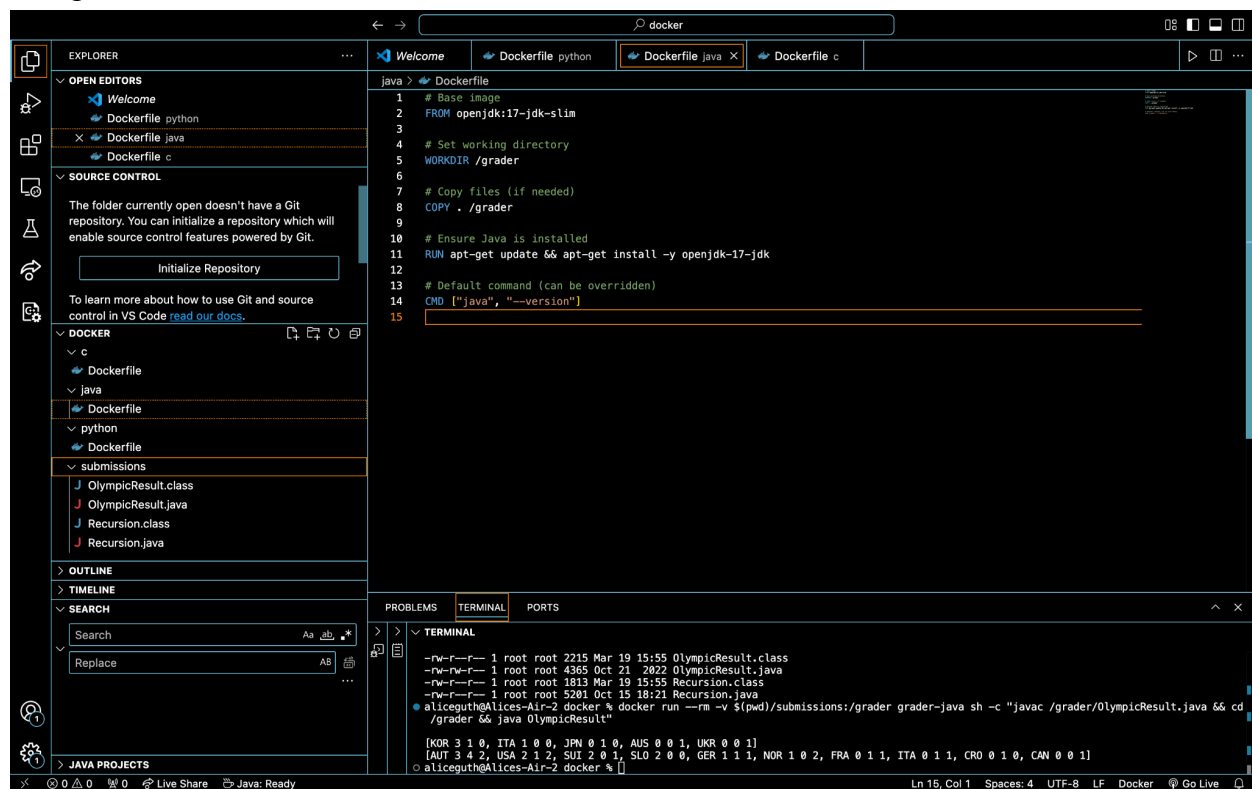
File storage:

The screenshot displays the AWS Management Console interface for an Amazon S3 bucket named 'grade-gator-files'. The left sidebar shows the navigation menu with options like 'General purpose buckets', 'Directory buckets', 'Table buckets', 'Access Grants', 'Access Points', 'Object Lambda Access Points', 'Multi-Region Access Points', 'Batch Operations', and 'IAM Access Analyzer for S3'. The main content area shows the bucket's details, including tabs for 'Objects', 'Metadata', 'Properties', 'Permissions', 'Metrics', 'Management', and 'Access Points'. The 'Objects' tab is active, showing a list of three folders: 'pdf-submissions/', 'student-submissions/', and 'test-folder/'. Above the list, there are buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'. A search bar is also present with the text 'Find objects by prefix'.

Name	Type	Last modified	Size	Storage class
pdf-submissions/	Folder	-	-	-
student-submissions/	Folder	-	-	-
test-folder/	Folder	-	-	-

Above is a screenshot of the current AWS S3 bucket for file storage. When a student or an instructor uploads a code/pdf/json file from their end, the file they submitted will automatically be stored to the corresponding folder. Code files submitted by students are stored inside the student-submission folder; PDF files are stored inside the pdf-submissions folder. A grading-rubric folder will be created once a json file is uploaded from the instructor mode. And the temporary test-folder will be removed after all the testing is done. Inside each folder, each file has its name, type, uploaded time, and size displayed, and we will also be able to download and open the file.

Autograder:



The above image shows the progress we've made on developing the autograder. On the sidebar, it shows we've created a container using Docker, as well as a Dockerfile, for C, Java, and Python. By creating these Dockerfiles, we can now run student code in each respective container. Additionally, there exists a submission folder, where we will store student assignments. Then, using the command shown in the terminal, Docker will run the specified student submission and print out the results of its Main function. The next step is connecting an instructor's JSON file which specifies the correct code behavior, with the output given by running the student file. After this connection, the grade the student received on this assignment should be outputted instead of the output from the Main method.

Current API:

```
"section": "string",
"department": "string",
"created_at": "2025-03-20T00:43:31.093Z",
"updated_at": "2025-03-20T00:43:31.093Z"
}
```

POST /api/courses/

courses_create

Create a new course

Parameters

Try it out

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "name": "string",
  "number": "string",
  "term": "string",
  "section": "string",
  "department": "string"
}
```

Responses

Unset

```
/api/
  /schema/           # API schema (OpenAPI)
  /docs/             # Swagger UI
  /redoc/            # ReDoc UI

# Accounts endpoints
/current-user/       # Get current user info

# Courses endpoints
/courses/            # List, create courses
/courses/{id}/       # Get, update, delete course
/students/           # List, create students
/students/{id}/      # Get, update, delete student
/instructors/        # List, create instructors
/instructors/{id}/   # Get, update, delete instructor

# Assignments endpoints
/assignments/        # List, create assignments
/assignments/{id}/   # Get, update, delete assignment
/submissions/        # List, create submissions
/submissions/{id}/   # Get, update, delete submission
```

```
# Grading endpoints
/grades/                # List, create grades
/grades/{id}/          # Get, update, delete grade
/feedback/             # List, create feedback
/feedback/{id}/        # Get, update, delete feedback
```

The screenshots above show the current hierarchy and apis being used on the front end. These will be updated substantially as we build the actual functionality of the website.

Tests

To test the front-end designs, we created Figma prototypes that users can interact with and click through. We tested our prototypes by demonstrating them in our Sprint Review meeting. Our customers mentioned some additional functionalities that we haven't yet added to our prototypes. They suggested we proceed with prototype one and first focus on the functionalities rather than the designs.

To test the autograder's success in running code in either Java, C, or Python, we created containers for each language using Docker, and then uploaded an old assignment from CSC 221 which was successfully run in the container, and the output from its main function was displayed appropriately.

For the file storage functionality, we set up an AWS S3 bucket and connected it to the current front-end. To test it, we added new submissions, attached different types of files, and confirmed the submissions. By checking the S3 bucket and successfully downloading and opening the files being uploaded, we confirmed the file storage system is working properly.

As we progress, we will be able to technically test the acceptance tests given in our proposal. Below, we outline the technical tests we plan to conduct.

Acceptance Tests from Proposal:

1. Given I am getting ready to grade an assignment, when I tell the platform I want to grade using standards based grading then it will determine how many standards each student achieved on the assignment.
 - a. To technically test this acceptance test, the instructor would input a student assignment and JSON file that contains the assignment rubric, which would contain the instructions for standard based grading, and then submit it to be graded. The output would be a score/which standards were met, which would

- confirm the acceptance test passed.
2. Given I am preparing to complete an assignment, when I open the assignment and navigate through the questions, then the rubric for each question should also appear alongside the question.
 - a. To technically test this acceptance test, the instructor would click on an assignment using the user interface, and navigate to view each question in the assignment. If the rubric for the assignment/question appears next to the question, the acceptance test passes.
 3. Given I am preparing to release an assignment to a class, when I am on the settings page to set the time allowed for the assignment and when it is due, then each student should be automatically given whatever additional time or other accommodations they require.
 - a. To technically test this acceptance test, the instructor would upload an assignment then navigate to the page where they can set the time to take, due date, etc. of the assignment. If the acceptance test passes, when the instructor sets a standard time to take, and then opens the assignment, students should have personalized due dates and time limits for the assignment on their view.
 4. Given I am getting ready to complete an assignment, when I open the homepage of the software, then I should be able to find the course I'm looking for quickly.
 - a. To technically test this acceptance test, the student could set a timer to navigate to the Grade Gator homepage, where all of their courses should be listed, and see how long it takes them to find the course they want. Then, depending on their standard of quickly, we can determine if the acceptance test passes or not.
 5. Given I am reviewing my student's work on a recent assignment, when I navigate to the results of the assignment then I will be able to see the distribution of who missed each question.
 - a. To technically test this acceptance test, the instructor should navigate to the assignment page after the due date has passed, click on the review assignment button, and then each question should be displayed alongside each person/how many people missed the question on the assignment. To pass this acceptance test, the instructor should be able to visually and easily see which questions the most people missed.

Project Management

Change log:

Date: Date the change was made	Description: A summary about the change made to the system
2/27	Repository created for backend

2/27	Repository created for frontend
3/2	Backend repository can connect to frontend
3/3	Create database containers for users, courses, and assignments and populate the database with needed attributes
3/5	Present basics course data on front end
3/6	Refine database design diagram based on frontend user needs
3/7	Begin prototypes for future homepage, course page, and assignment page, designed with autograder integration in mind
3/9	Containers for java, python, and c that run code files and return the output in Docker
SPRING BREAK	
3/15	Successfully upload and store files locally
3/16	Standardize the frontend tech stack
3/16	Successfully upload and store files through the front end to AWS
3/17	Complete interactive prototypes for student and professor view of homepage, course pages, and assignment pages
3/19	Meet with Computer Science professors for Sprint Review and feedback

Sprint Review

The sprint review occurred on March 19th with a customer, Dr. Lim, stakeholders, professors who will use Grade Gator, and team members.

We presented, in prototype form, the goal of the user navigating through different pages on the future Grade Gator website. We focused on the organization and flows between pages for both the student view and professor view. We also presented the user story of a student uploading a

file and it being securely stored in the database and visible in the future. At the time of the review, the “student” was more vague and was presented as Ann uploading a file locally and it being securely stored in her personal AWS server. We also shared a progress report on the goal of implementing an autograder that would take submitted files and compare it to JSON files uploaded by professors. At the time of the review, Alice reported that the components of the autograder had been created and now need to be connected to the front and back end.

The customers were able to provide key feedback about the front end prototype. Dr. Lim approved our third prototype for implementation. Dr. Williams saw features in the other prototypes that she would like to include in the final product, specifically, the ability to upload rubrics. Dr. Lim emphasized that he would like to see a usable front end as soon as possible, which has led us to refine our front end development process. We will plan to implement a basic front end design in the next sprint, which can be refined in the future. In terms of overall product, Dr. Williams emphasized her continued desire for standards-based grading features in the final product and Dr. Heyer brought up support for PDF assignments and multiple choice quizzes, which are common needs in the Math Department. At the moment, all the software is hosted locally, and Dr. Lim has voiced that he feels we could continue to host locally until the end of the semester, at which point he will work on the migration to open servers.

Creating the basic database structure, setting up file storage through AWS, and setting up Docker containers for the autograder to use, were the most valuable features we provided during this sprint. Creating this basic functionality and structure will put us in a good position to advance what we have currently and expand the functionality of our product.

Our team’s updates on the backend development seemed to provide the most value to the customer during the sprint because it informed them of foundational development decisions we have made. For instance, the decision to have different front and back end repositories or the choice of using react for front end development will constrain our future development processes. Since we do not have a fully connected software yet, explaining each piece was important to reassure the customer that we are making progress.

There was a gap in expectations for features that would be included in the sprint. We decided as a team to focus on the autograding function first, then expand to support standards based grading and other assignment options. The customer’s expressed continued interest in these alternative grading methods, which are in the overall product backlog, but were not addressed in this sprint.

The customers deepened our understanding of how they expected the rubric feature to behave. We initially believed the customers wanted an option to upload a rubric in PDF form. Prompted by our prototypes, our customers clarified that the rubric they wanted visible to students was the

one manually entered in a GUI for the autograding or traditional grading on the website. They also expressed interest in the option to select which parts of the rubric are available to students.

The delivered work aligned fairly well with the customer's initial expectations and priorities. There was a disconnect between the level of UI design the customer expected our team's software to have, so the customer clarified their focus on working software over well-designed software. Otherwise, the customer was pleased with the foundational work that had been done during the sprint.

It is difficult to express the customer's perception of Grade Gator's quality at this point because the back and front end are not completely connected at this point. However, the customers seemed confident in our team's development decisions, such as using AWS for secure database storage and Docker for the autograder. The sample file submissions and assignment creations hosted locally and stored through AWS had low ease of use, but that was acknowledged by our team and it is a goal to improve in the next iteration.

Our goal for the next sprint is to have working software (most likely hosted locally) which will make the delivery process smoother. We were able to present part of our review with an overhead projector, and we would like to have enough working pieces to present all of our review on a projector for the next sprint. This way, customers have visuals and can provide more specific feedback. We could improve our presentation outline and give a brief introduction including that outline next time, since we just jumped into information for this sprint review.

There was slight miscommunication on the goal surrounding the user interface. We set our goal to create numerous designs for the user interface so that we could get user feedback before implementing a design in code. However, the customer expected us to have an interface already functioning. So, we have made this our top priority for the next sprint. We also did not get full functionality of the autograder, in terms of grading an assignment in comparisons to a rubric or set of test cases. But, we received clarification from the customer on how the rubric and grading functionality worked for the autograder and have this functionality at the end of the next sprint.

Sprint Retrospective

DuBose was the scrum master who organized tasks for this iteration. He also jump started the project by creating basic front end and back end servers. Maddie became the product owner and worked on front end design for professor and student views. Katherine and Ally also worked on front end design, creating Figma prototypes and acquiring user feedback. David gathered materials about front end techstacks to help Maddie, Katherine, and Ally create the webpage in the next iteration. This will ensure we have standardized front end technologies. Alice created the autograder, while Wendy and Ann worked on file storage. Therefore, if a student uploads a PDF or code file, we can store it and access it in our database.

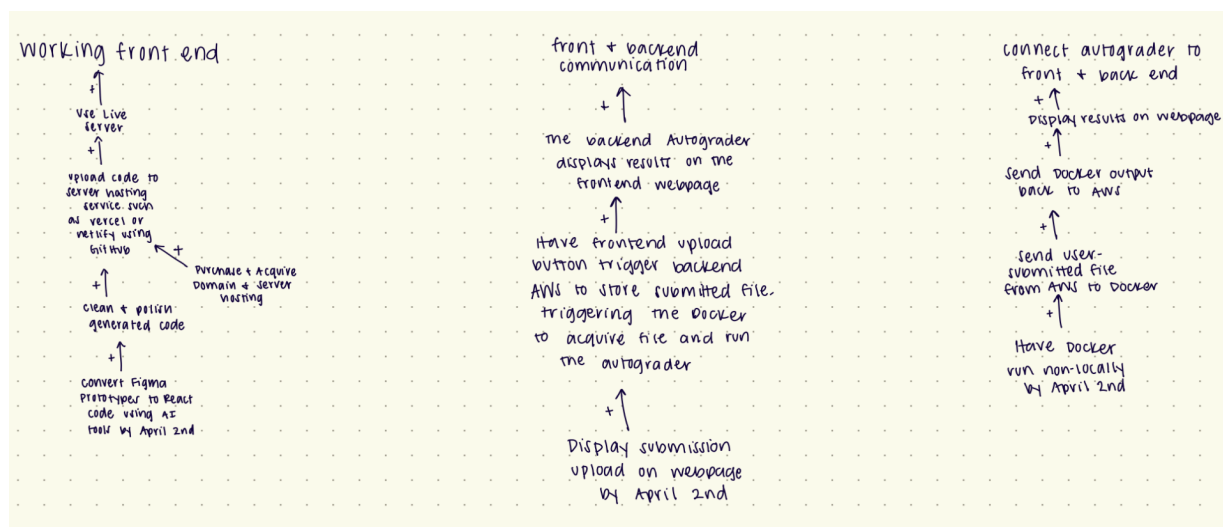
A large challenge for us is finding times to meet. Because our group is so large and we have different schedules, there are not many times where we can all be present. Another challenge was assigning tasks based on different skill levels and experiences. Most of us have more front end experience, but this project requires knowledge of back end and database programming. Similarly, brainstorming tasks and knowing where to begin the project was a challenge.

To overcome the challenge of finding meeting times, we have broken into groups where we focus on different parts of the project. With smaller groups, we are able to find meeting times easier. Along with this, we all communicate often over slack to work well asynchronously. To address varying skill levels, team members with specific experience in certain areas have written documentation for the development tools we are using. For example, David has outlined information on front end development for us to consult.

The advice a close third party would give our team is to continue including abundant documentation in our individual projects. Since the team is large and each team member is often doing a unique subtask, it is important to share information with the whole team. Further, we are all learning new technologies as we go, so it is important that team members do not get lost in updates. It is also important as we begin to merge pieces together to have a very specific plan to follow so that everyone is clear on what the big picture is.

Goals for the Next Iteration

The three main goals for the next iteration is to develop a working front end, achieve communication between the front and backend, and connect the autograder to the front and back end. We have created a SMART goal hierarchy below to help us achieve these goals.



Product Backlog:

- Create distinct user interfaces for student and instructor views.
- Create a gradebook that supports standards based grading
- Create a submission service that included the use of an autograder
- Set up a basic autograder that connects to Docker containers, finds student code submissions and runs alongside a JSON file the instructor has uploaded with the assignment rubric.
- Host the frontend on a website rather than a local host.
- Organize the backend databases so necessary information is stored and can be easily accessed.
- Require login authentication.
- Develop standards based grading ability
- Create GUI for instructors to create assignment rubric
- Research and implement ways to securely store student data once on a shared server.
- Include review of instructor comments and grading on the student view of assignments.
- Research the potential for Moodle integration of class rosters.
- Allow students to view rubrics for assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.
- Meet with professors in the math department to discuss needs of department assignments.

Sprint Backlog:

1. Implement Figma design on web server using react
2. Host site on server
3. Connect front and back end
4. Connect autograder to front and back end
5. Update database and user interface to present necessary course data
6. Clean up backend code files and API structure to be more consistent, document API endpoints format.